

Linear Regression

→ Easy to Understand

→ Foundation to other ML algorithms

It is a supervised ML algorithm.

Regression classification

1) Linear Regression

a) Simple Linear Regression → 1 Input / 1 Output

b) Multiple LR → >1 Input / 1 Output

c) Polynomial LR → Works if data is non linear

Example

Independent
Feature
↓

cgpa

7.1

4.7

8.9

8.1

⋮

Dependent
Feature
↓

package

3.5

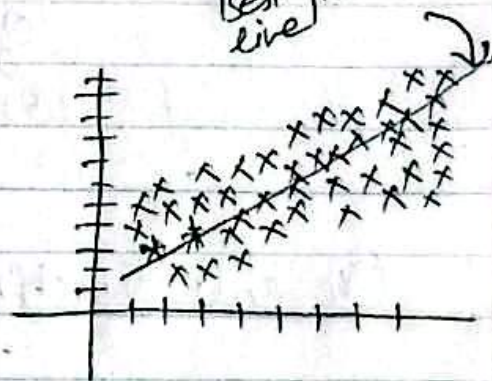
1.2

4.2

3.9

⋮

Best fit
line



Why the data is not linear?

⇒ Real world dataset is affected by real world factors

Equation of line ⇒ $y = mx + c$

where, m → slope

c = y-intercept

x = Real data (Input)

⊗ What is Best fit line ?

⇒ The LR algorithm draws a straight line that try to pass through every data points which has minimum error.

The best fit line pass closely through every data points.

⊗ Formula found in Research Paper

$$h_0(x) = \theta_0 + \theta_1 x$$

where, $h_0(x)$ = Predicted Value

x = Input Feature

θ_0 = Y Intercept

θ_1 = Slope

⊗ Cost Function

$$J(\theta_0, \theta_1) = \frac{1}{n} \sum (y_i - h_0(x_i))^2 \quad \begin{matrix} \rightarrow \text{predicted} \\ \text{actual} \end{matrix} \quad [\text{MSE} \rightarrow \text{Mean Squared Error}]$$

Low Cost Function $\approx$ Better Model

⊗ Final aim $\Rightarrow$ In ada to get best fit line, minimize $J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (y_i - h_0(x_i))^2$

By changing value of θ_0 and θ_1 .

Dataset

x	y
1	1
2	2
3	3

Let consider, $\theta_0 = 0$ and $\theta_1 = 1$

$\therefore$ if $x=1$, $h_0(x) = 1 \times 1 = 1$
 if $x=2$, $h_0(x) = 1 \times 2 = 2$
 if $x=3$, $h_0(x) = 1 \times 3 = 3$

} predicted points

$$\therefore J(\theta_1) = \frac{1}{n} \sum_{i=1}^n (y_i - h_0(x_i))^2$$

$$= \frac{1}{3} [(1-1)^2 + (2-2)^2 + (3-3)^2] = 0$$

Let, $\theta_1 = 0.5$,

if $x=1$, $h_0(x) = 0.5$

if $x=2$, $h_0(x) = 1$

if $x=3$, $h_0(x) = 1.5$

$$\therefore J(\theta_1) = \frac{1}{3} [(1-0.5)^2 + (2-1)^2 + (3-1.5)^2]$$

$$= 1.16$$

Let $\theta_1 = 0$,

if $x=1$, $h_0(x) = 0$

if $x=2$, $h_0(x) = 0$

if $x=3$, $h_0(x) = 0$

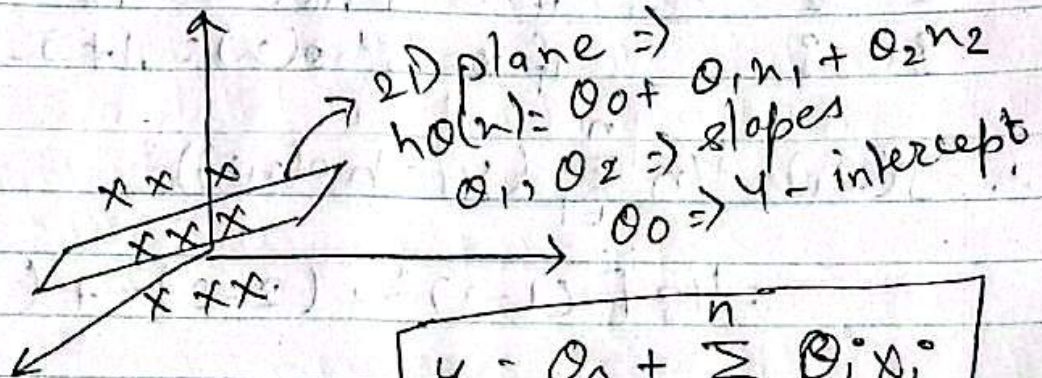
$$\therefore J(\theta) = \frac{1}{3} [(1-0)^2 + (2-0)^2 + (3-0)^2]$$

$$= 4.66$$

⊗

Multiple Linear Regression

⇒ More than one input features

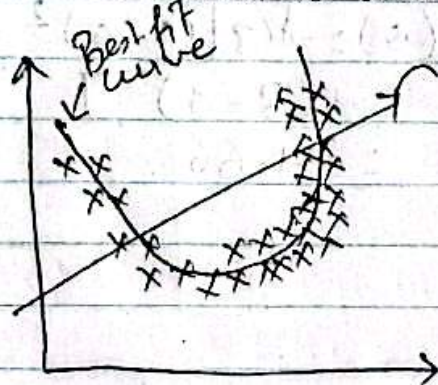


$$y = \theta_0 + \sum_{i=1}^n \theta_i x_i$$

⊗

Polynomial Regression

→ Not every data has linear relation. The relation between some data can be defined by curves.



$h_0(x) = \theta_0 + \theta_1 x_1$
 Error $\uparrow \uparrow$ (very high)

Polynomial degrees

0, $h_0(x) = \theta_0 + x_i \Rightarrow$ Constant

1, $h_0(x) = \theta_0 x_i + \theta_1 x_i \rightarrow$ SLR

2, $h_0(x) = \theta_0 x_i^2 + \theta_1 x_i + \theta_2 x_i^2$

3, $h_0(x) = \theta_0 x_i^3 + \theta_1 x_i^2 + \theta_2 x_i + \theta_3 x_i^3$

$\vdots$

$n, h_0(x) = \theta_0 x_i^0 + \theta_1 x_i^1 + \theta_2 x_i^2 + \theta_3 x_i^3 + \dots + \theta_n x_i^n$

Simple Polynomial Regression

Multiple Polynomial Regression

$$h_0(x) = \theta_0 + \sum \theta_{i_1, i_2, \dots, i_n} x_1^{i_1} x_2^{i_2} \dots x_n^{i_n}$$

Ex Two features upto degree 2

$$h_0(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2$$

For degree 3,

$$h_0(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2 + \theta_6 x_1^2 x_2 + \theta_7 x_1 x_2^2 + \theta_8 x_1^3 + \theta_9 x_2^3$$

Ex 3 features degree 2,

$$h_0(x) = \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_1^2 + \theta_5 x_2^2 + \theta_6 x_3^2 + \theta_7 x_1 x_2 + \theta_8 x_1 x_3 + \theta_9 x_2 x_3$$

Regression Matrix

Performance Metrics

1) MAE (Mean Absolute Error)

→ Average of the absolute difference between predicted and actual values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Actual value Predicted value

2) MSE (Mean Squared Error)

→ Average of squared errors between actual and predicted values.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

3) Root Mean Squared Error (RMSE)

→ Square root of MSE; gives error in original unit

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

4) R^2 score

→ Measures how much variance in the target variable is explained by the model.

closer to 1 means better performance.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

$$\text{where, } SS_{res} = \sum (y_i - \hat{y}_i)^2$$

$$SS_{tot} = \sum (y_i - \bar{y})^2$$

5) Adjusted R^2

→ Used when there are many features. It is more reliable for multiple regression.

$$\text{Adjusted } R^2 = 1 - (1 - R^2) \frac{n-1}{n-k-1}$$

where, n = samples

k = no. of features

Why use Performance Metrics?

- 1) To measure how accurate the model is
- 2) To compare multiple models
- 3) To improve model iteratively

In short, performance metrics tells how good your model is

Regularization

Regularization is a technique used in ML to reduce overfitting and improve model performance.

How regularization works?

→ Regularization adds a penalty term to the cost function (loss function).

Without Regularization,

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

With regularization,

$$J(\theta) = \text{Loss} + \text{Penalty}$$

Penalty discourages large weights, reduces overfitting.

⊗ Regularization Techniques

1) L1 Regularization (Lasso Regularization)

Penalty :- $\lambda \sum_{j=1}^n |\theta_j| \Rightarrow \lambda [|\theta_1| + |\theta_2| + |\theta_3| + \dots + |\theta_n|]$
hyperparameter λ ← slope

where, θ_j = model weight

n = no. of features

λ = Regularization strength (Hyperparameter)

2) L2 Regularization (Ridge Regression)

Penalty!:- $\lambda \sum_{j=1}^n \theta_j^2 \Rightarrow \lambda [\theta_1^2 + \theta_2^2 + \theta_3^2 + \dots + \theta_n^2]$

hyperparameter λ slope

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n \theta_j^2$$

$\Rightarrow$ It adds squared values of weights to the loss function from sklearn.linear_model import Ridge

3) Elastic Net Regularization

$\rightarrow$ It is the combination of L1 + L2 penalties
 $\rightarrow$ Good when features are correlated.

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \left(\alpha \sum_{j=1}^m |\theta_j| + (1-\alpha) \sum_{j=1}^m \theta_j^2 \right)$$

where,

λ = Overall Regularization strength

$\alpha \in [0, 1]$ = mix ratio between L1 & L2

θ_j = weight of feature j

In other word,

$$\text{Cost fn} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda_1 \sum (\text{slope})^2 + \lambda_2 \sum |\text{slope}|$$

$\downarrow$ $\downarrow$ $\downarrow$
MSE Reduces Feature
 Overfitting selection

$\lambda_1, \lambda_2 \Rightarrow$ Hyperparameter Tuning

Quick Intuition:-

- 1) Lasso:- Picks only the important features
- 2) Ridge:- Keeps all the features but shrinks their influence
- 3) Elastic Net:- Best of both — shrinks + select features if correlated (Multicollinearity)
 $\hookrightarrow$ Positively Correlated

Note:- Ridge focuses on reducing overfitting

Lasso focuses on feature selection

Elastic Net ($L_1 + L_2$) focuses on reducing overfitting while performing feature selection

Pipeline

A pipeline is a way to chain multiple data processing steps and modeling steps into a single workflow.

Typical steps in a Pipeline :-

- 1) Data Preprocessing $\Rightarrow$ Scaling, Encoding, Missing Values Handling
- 2) Feature Engineering $\Rightarrow$ Polynomial features, PCA, feature selection
- 3) Model Training $\Rightarrow$ ML Algorithms
- 4) Predictions $\Rightarrow$ Apply model to new data

Advantages

- 1) Avoids data leakage
- 2) Cleaner code
- 3) Easy hyperparameter tuning
- 4) Consistent workflow

Disadvantages

- 1) Less flexible
- 2) Debugging can be tricky
- 3) Complex for small tasks
- 4) limited to compatible libraries